Generalized N×N Matrix Engine for FPGA Neural Networks

Complete Project Specification and Development Guide

Version: 1.0
Date: December 2024
Author: FPGA Neural Network Project
Target: Artix-7 FPGA (Scalable to larger devices)

Table of Contents

1. Executive Summary
2. Project Background
3. Design Philosophy
4. Fixed-Point Arithmetic
5. Architecture Overview
6. Subsystem 1: Compute Subsystem
7. Subsystem 2: Memory Subsystem
8. Subsystem 3: Postprocess Subsystem
9. Subsystem 4: Control Subsystem
10. Top Level Integration
11. Operations and Data Flow
12. Zero Padding Strategy
13. Development Plan
14. Testing Strategy
15. Resource Estimates
16. Performance Analysis
17. Future Enhancements
18. Appendices

---

1. Executive Summary

1.1 Project Goal

Design and implement a **generalized N×N matrix computation engine** in VHDL that accelerates neural network training and inference on FPGAs. The engine supports variable-dimension matrix operations through zero-padding, enabling efficient processing of different layer sizes without hardware reconfiguration.

1.2 Key Features

Feature	Description
Generalized Dimensions	Single 8×8 (or 32×32) engine handles any smaller dimensions
Three Operations	MatVec (forward), MatVec_T (backward), OuterProduct (update)
Parallel Processing	All N rows compute simultaneously
Pipelined Design	Overlapped computation stages for throughput
Configurable Activation	Linear, ReLU, Leaky ReLU
Fixed-Point Arithmetic	Q4.11 format with 48-bit accumulators

1.3 Performance Target

Metric	Sequential Design	Matrix Engine	Improvement
Cycles per sample	210	70-105	**2-3× faster**
Forward pass (4-8-4-1)	110 cycles	38 cycles	**2.9× faster**
MAC utilization	7.7%	50-100%	**6-13× better**

1.4 Target Applications

- Binary classification (Iris dataset)
- Multi-layer perceptrons (MLPs)
- Small to medium neural networks (up to 32 neurons per layer)
- Real-time inference and on-chip training

---

2. Project Background

2.1 Current Implementation Status

The existing FPGA neural network implementation uses a **sequential single-neuron architecture**:

```
Current Architecture: 4-8-4-1 Network
├── Layer 1: 8 neurons × 4 inputs = 32 weights
├── Layer 2: 4 neurons × 8 inputs = 32 weights
├── Layer 3: 1 neuron × 4 inputs = 4 weights
└── Total: 68 weights + 13 biases = 81 parameters
```

Current Performance Issues

1. **Sequential Processing**: Neurons process one input at a time
2. **Low MAC Utilization**: Only one multiplication per cycle
3. **High Cycle Count**: ~210 cycles per training sample
4. **Inefficient Scaling**: Adding neurons linearly increases time

Existing Bugs Fixed

- ✅ Bias gradient scaling error (8192× slower learning)
- ✅ Dataset ordering problem (shuffling implemented)
- ✅ Dying ReLU (He initialization + positive bias)
- ✅ Update pass timeout (latching logic)
- ✅ FSM out-of-bounds indexing

Current Accuracy

- Training: 54.17% (13/24 samples)
- Test: 33.33% (2/6 samples)
- Issue: Converges to trivial solution (always predict class 1)

2.2 Why Matrix Engine?

The matrix engine addresses fundamental architectural limitations:

Problem	Current Design	Matrix Engine Solution
Sequential MACs	1 MAC/cycle	N^2 MACs/cycle
Input reloading	Reload for each neuron	Broadcast to all rows
Weight access	One weight at a time	Full row parallel read
Scalability	Linear time increase	Constant time for $\leq N$

2.3 Design Constraints

Constraint	Value	Rationale
Target FPGA	Artix-7 35T	Entry-level development board
DSP Slices	≤ 90	Device limit
LUTs	≤ 20,800	Device limit
Block RAM	≤ 50 (18Kb)	Device limit
Clock Frequency	100 MHz	Conservative timing
Data Format	16-bit fixed	Balance precision/resources

---

3. Design Philosophy

3.1 Subsystem-Based Hierarchy

The design is organized into **four independent subsystems**, each containing multiple sub-modules:

```
matrix_engine_top
│
├── SUBSYSTEM 1: compute_subsystem
│   └── Contains all multiply-accumulate logic
│
├── SUBSYSTEM 2: memory_subsystem
│   └── Contains all data storage
│
├── SUBSYSTEM 3: postprocess_subsystem
│   └── Contains bias, activation, saturation
│
└── SUBSYSTEM 4: control_subsystem
    └── Contains FSM and configuration
```

Why This Structure?

1. **Independent Testing**: Each subsystem can be verified in isolation
2. **Clear Interfaces**: Well-defined ports between subsystems
3. **Easier Debugging**: Isolate bugs to specific subsystem
4. **Modular Development**: Teams can work on different subsystems
5. **Reusability**: Subsystems can be reused in other projects

3.2 No Separate Package

Decision: Fixed-point constants are defined at the top level and passed down via generics, rather than using a separate VHDL package.

Rationale:

- Avoids package dependency issues
- Easier to modify parameters
- Cleaner synthesis reports
- More portable across tools

3.3 Generalization via Generics

All modules use generic parameters for configurability:

```
Generic Parameters:
├── MAX_DIM (8 or 32) - Maximum matrix dimension
├── DATA_WIDTH (16) - Operand bit width
├── ACCUM_WIDTH (48) - Accumulator bit width
└── FRAC_BITS (11) - Fractional bits for Q4.11
```

3.4 Weight-Stationary Dataflow

The engine uses **weight-stationary** architecture:

- Weights are loaded once and remain in registers
- Inputs stream through the MAC array
- Outputs are collected after all inputs processed

Benefits:

- Minimizes weight memory bandwidth
- Weights stay in fast register storage
- Simple control logic

3.5 Zero-Padding for Generalization

Smaller dimensions (e.g., 4×3) are handled by **zero-padding** to the maximum dimension (8×8):

- Unused weight positions are zero
- Unused input elements are zero
- Zero × anything = 0, so padded elements don't affect results
- Control logic masks unused outputs

---

4. Fixed-Point Arithmetic

4.1 Format Selection: Q4.11

After analysis of range and precision requirements, **Q4.11** format was selected:

Property	Value
Total Bits	16
Sign Bit	1
Integer Bits	4
Fractional Bits	11
Range	[-16.0, +15.9995]
Resolution	1/2048 ≈ 0.000488
Unity (1.0)	2048
Maximum	32767
Minimum	-32768

Why Q4.11 Over Q2.13?

Criterion	Q2.13	Q4.11	Winner
Range	[-4, +4)	[-16, +16)	Q4.11
Precision	0.000122	0.000488	Q2.13
Accumulation headroom	Low	High	Q4.11
Overflow risk	High	Low	Q4.11

Decision: Q4.11 provides better range for matrix accumulation where sums can grow large.

4.2 Accumulator Format: Q14.33

To prevent overflow during MAC operations:

Property	Value
Total Bits	48
Sign Bit	1
Integer Bits	14
Fractional Bits	33

Bit Growth Analysis

```
Input A: Q4.11 (16 bits)
Input B: Q4.11 (16 bits)
Product A×B: Q8.22 (32 bits)

Sum of N products: Q8.22 + log2(N) integer bits
For N=64 (8×8): Q14.22 minimum

Using Q14.33 provides:
- 11 extra fractional bits for precision
- Headroom for bias addition
```

4.3 Saturation and Rounding

Saturation Rules

```
If value > +32767: saturate to +32767, set overflow flag
If value < -32768: saturate to -32768, set overflow flag
Otherwise: keep value as-is
```

Rounding (Round-to-Nearest)

```
Before shifting right by N bits:
  Add (1 << (N-1)) to implement rounding
  Then arithmetic shift right by N bits
```

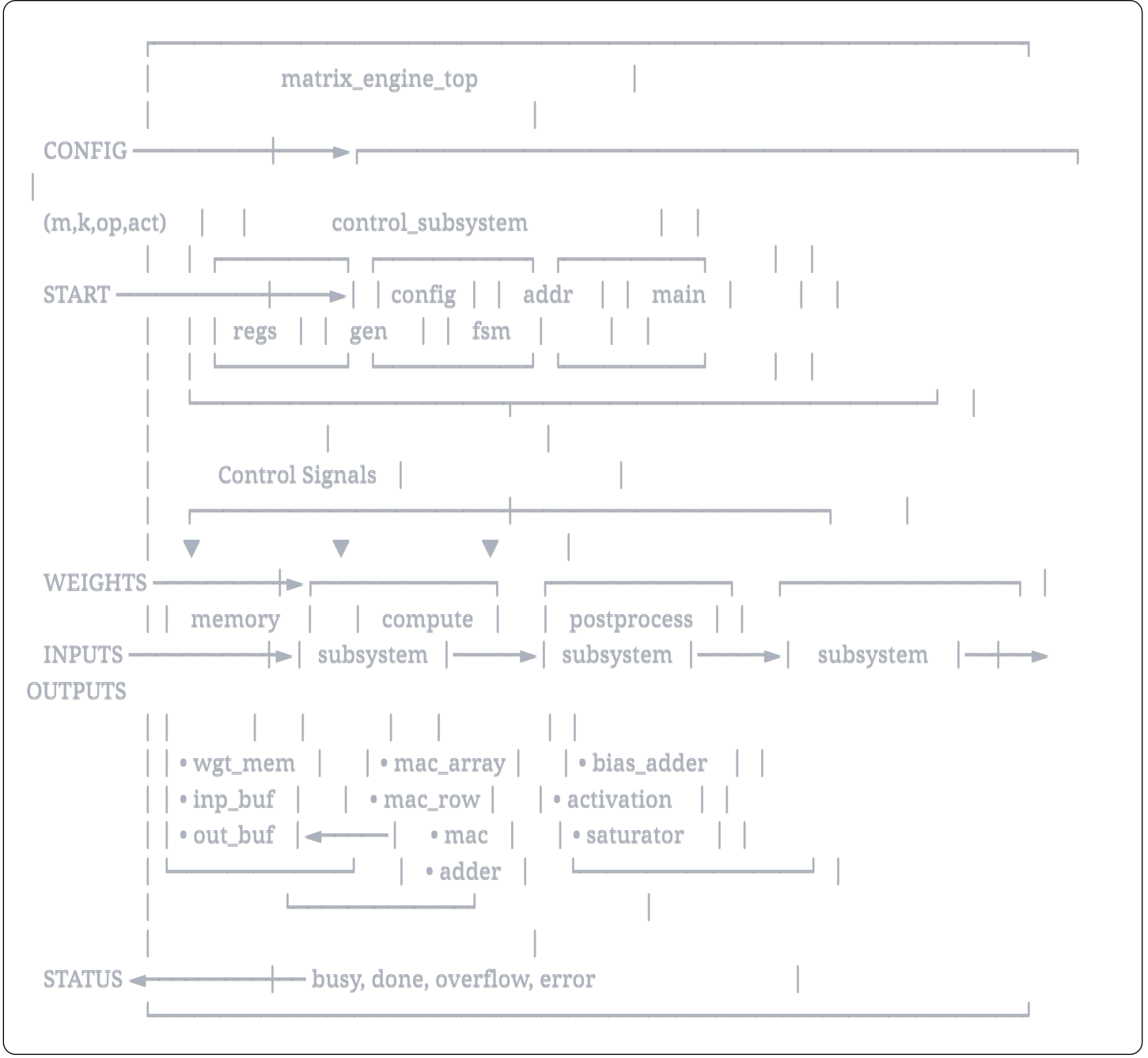
4.4 Key Constants

Constant	Name	Value	Usage
2048	FP_ONE	1.0 in Q4.11	Comparisons, derivatives
32767	FP_MAX	Maximum positive	Saturation limit
-32768	FP_MIN	Minimum negative	Saturation limit
1024	ROUND_CONST	0.5 LSB	Rounding before shift
11	FRAC_BITS	Fractional bits	Shifting operations

---

5. Architecture Overview

5.1 Top-Level Block Diagram



5.2 Module Count Summary

Subsystem	Sub-Modules	Total Instances (8×8)
compute_subsystem	4 types	1 array + 8 rows + 64 MACs + 8 adders
memory_subsystem	3 types	1 weight + 1 input + 1 output buffer
postprocess_subsystem	3 types	1 bias + 1 activation + 1 saturator
control_subsystem	3 types	1 config + 1 addr_gen + 1 FSM
TOTAL	**13 types**	**~90 module instances**

5.3 Data Width Summary

Signal Type	Width	Format	Description
Input data	16	Q4.11	Features, activations
Weights	16	Q4.11	Trainable parameters
Bias	16	Q4.11	Per-neuron offset
Product	32	Q8.22	Weight × Input
Accumulator	48	Q14.33	Sum of products
Output	16	Q4.11	Final result

5.4 Signal Flow Summary

```
1. Configuration Phase:
   External → config_regs → All subsystems

2. Weight Loading Phase:
   External → weight_memory (stored)

3. Compute Phase:
   input_buffer → mac_array → postprocess → output_buffer

4. Output Phase:
   output_buffer → External
```

---

6. Subsystem 1: Compute Subsystem

6.1 Purpose

The compute subsystem performs all **multiply-accumulate (MAC) operations**. It is the computational heart of the matrix engine, containing the parallel MAC array and reduction logic.

6.2 Hierarchy

```
compute_subsystem
│
├── mac_array (1 instance)
│   │
│   ├── mac_row [0] (1 of N instances)
│   │   ├── mac_unit [0..N-1] (N instances per row)
│   │   └── adder_tree (1 per row)
│   │
│   ├── mac_row [1]
│   │   └── (same structure)
│   │
│   └── mac_row [N-1]
│       └── (same structure)
│
Total: N² mac_units + N adder_trees + N mac_rows + 1 mac_array
For N=8: 64 + 8 + 8 + 1 = 81 module instances
```

6.3 Module Descriptions

6.3.1 mac_unit

Purpose: Single multiply-accumulate unit. The fundamental computational element.

Function:

- Multiplies 16-bit weight by 16-bit input to produce 32-bit product
- Sign-extends product to 48 bits
- Adds to running accumulator (or clears accumulator)
- Outputs 48-bit accumulated result

Key Behaviors:

- On `clear`: Accumulator resets to zero
- On `enable`: Accumulator += weight × input
- Otherwise: Accumulator holds value

Timing: 1 cycle latency, 1 MAC per cycle throughput

Interfaces:

- Inputs: clk, rst, clear, enable, weight[15:0], data_in[15:0]
- Outputs: accum[47:0]

6.3.2 adder_tree

Purpose: Reduces N accumulator values to a single sum using binary tree structure.

Function:

- Takes N (typically 8) 48-bit inputs
- Performs pairwise addition in tree structure
- Produces single 48-bit sum
- Optional pipeline registers for timing closure

Tree Structure (N=8):

```
Level 1: 4 adders (in[0]+in[1], in[2]+in[3], in[4]+in[5], in[6]+in[7])
Level 2: 2 adders (sum of above pairs)
Level 3: 1 adder (final sum)
```

Timing:

- Combinational: 0 cycles latency
- 1 pipeline stage: 1 cycle latency
- 2 pipeline stages: 2 cycles latency

Interfaces:

- Inputs: clk, rst, data_in[N×48-1:0], valid_in
- Outputs: sum_out[47:0], valid_out

6.3.3 mac_row

Purpose: Contains N MAC units computing dot product of one weight row with input vector, plus adder tree for reduction.

Function:

- Receives one row of weights (N values)
- Receives input vector (N values, broadcast)
- Each MAC unit computes weight[i] × input[i]
- Adder tree sums all N MAC outputs
- Supports partial row activation via k_dim

Zero-Padding Support:

- MACs with index >= k_dim are disabled
- Disabled MACs maintain accumulator at 0
- Result is mathematically correct for smaller dimensions

Timing: K cycles to accumulate + adder tree latency

Interfaces:

- Inputs: clk, rst, clear, enable, weights[N×16-1:0], inputs[N×16-1:0], k_dim[5:0]
- Outputs: row_result[47:0], valid_out

6.3.4 mac_array

Purpose: Top module of compute subsystem. Contains N mac_rows operating in parallel.

Function:

- Instantiates N mac_row modules
- Broadcasts input vector to all rows
- Provides different weight row to each mac_row
- Collects N results simultaneously
- Supports partial array activation via m_dim

Parallelism: All N rows compute simultaneously, processing one input element per cycle across all rows.

Zero-Padding Support:

- Rows with index >= m_dim are disabled
- Results from disabled rows are ignored
- Only m_dim results are valid

Timing: K cycles for MAC + adder tree latency

Interfaces:

- Inputs: clk, rst, clear, enable, weights_flat[N^2×16-1:0], input_vector[N×16-1:0], m_dim[5:0], k_dim[5:0]
- Outputs: results[N×48-1:0], results_valid[N-1:0]

6.3.5 compute_subsystem (Wrapper)

Purpose: Wraps mac_array and provides clean interface to other subsystems.

Function:

- Instantiates single mac_array
- Manages control signals (clear, enable)
- Generates combined output_valid signal
- Provides busy indication

Interfaces:

- Inputs: clk, rst, clear, enable, weight_data[N^2×16-1:0], input_data[N×16-1:0], m_dim[5:0], k_dim[5:0]
- Outputs: output_data[N×48-1:0], output_valid, busy

---

7. Subsystem 2: Memory Subsystem

7.1 Purpose

The memory subsystem handles all **data storage**: weight matrices, input buffering, and output buffering. It provides the data interface between external world and compute engine.

7.2 Hierarchy

```
memory_subsystem
│
├── weight_memory (1 instance)
│   └── Stores W, W^T, and bias
│
├── input_buffer (1 instance)
│   └── Serial-to-parallel conversion
│
└── output_buffer (1 instance)
    └── Parallel-to-serial conversion

Total: 3 module instances
```

7.3 Module Descriptions

7.3.1 weight_memory

Purpose: Stores weight matrix, transposed weights, and bias vector. Provides parallel row read capability.

Memory Organization:

```
Address Range    Content          Size (N=8)
═══════════════════════════════════════════
0x00 - 0x3F      W[row][col]      64 words
0x40 - 0x7F      W^T[row][col]    64 words
0x80 - 0x87      B[neuron]        8 words
═══════════════════════════════════════════
Total: 136 words × 16 bits = 2,176 bits
```

Function:

- **Write Path**: Accepts single weight value with address; automatically computes and writes transposed value
- **Read Path**: Reads entire row (N values) in parallel in single cycle
- **Transpose Support**: Separate storage for W^T eliminates transpose computation at runtime
- **Bias Access**: Direct indexing for bias values

Key Behaviors:

- On write: Stores to W[addr] and W^T[transposed_addr] simultaneously
- On read: Outputs entire row based on row index and transpose flag
- Bias: Separate read port for bias values

Timing:

- Write: 1 cycle per weight
- Read: 1 cycle for entire row (combinational read)

Interfaces:

- Inputs: clk, rst, write_en, write_addr[7:0], write_data[15:0], write_bias, read_row[5:0], read_transpose, read_bias_row[5:0]
- Outputs: weight_row[N×16-1:0], bias_out[15:0], ready

7.3.2 input_buffer

Purpose: Captures streaming input elements and provides parallel output to compute subsystem.

Function:

- Receives input elements one at a time (serial)
- Stores in internal buffer at specified index
- Counts received elements
- Signals completion when k_dim elements received
- Outputs all elements in parallel (zero-padded for unused positions)

Zero-Padding:

- Positions >= k_dim output as zero
- Ensures mathematically correct results for smaller dimensions

Key Behaviors:

- On valid input: Store at buffer[index], increment count
- On clear: Reset buffer to zeros, reset count
- Complete signal: Asserted when count equals k_dim

Timing: K cycles to fill buffer

Interfaces:

- Inputs: clk, rst, clear, data_in[15:0], index[5:0], valid, k_dim[5:0]
- Outputs: data_out[N×16-1:0], ready, complete, count[5:0]

7.3.3 output_buffer

Purpose: Stores computed results and provides serial streaming output.

Function:

- Receives all results in parallel (from postprocess)
- Stores in internal buffer
- Outputs one element at a time (serial)
- Tracks current output index
- Signals last element

Key Behaviors:

- On load: Store all parallel inputs, set count to m_dim
- On read_en: Output current element, advance pointer
- Last signal: Asserted on final element
- Empty/ready: Buffer state indicators

Timing: M cycles to stream all outputs

Interfaces:

- Inputs: clk, rst, data_in[N×16-1:0], load, m_dim[5:0], read_en
- Outputs: data_out[15:0], index[5:0], valid, last, empty, ready

---

7.3.4 memory_subsystem (Wrapper)

Purpose: Wraps all memory modules with unified interface.

Function:

- Instantiates weight_memory, input_buffer, output_buffer
- Routes signals to appropriate sub-module
- Provides unified interface to control subsystem

Interfaces: Combined interfaces of all three sub-modules (see full specification)

---

8. Subsystem 3: Postprocess Subsystem

8.1 Purpose

The postprocess subsystem processes MAC array outputs through a **three-stage pipeline**: bias addition, activation function, and output saturation.

8.2 Hierarchy

```
postprocess_subsystem
│
├── bias_adder (1 instance)
│   └── Stage 1: Add bias vector
│
├── activation_unit (1 instance)
│   └── Stage 2: Apply activation function
│
└── output_saturator (1 instance)
    └── Stage 3: Saturate to output format

Total: 3 module instances
Pipeline latency: 3 cycles
```

8.3 Module Descriptions

8.3.1 bias_adder

Purpose: Adds bias vector to accumulated results.

Function:

- Receives N 48-bit accumulated values
- Receives N 16-bit bias values
- Scales bias to match accumulator format (shift left by FRAC_BITS)
- Adds scaled bias to each accumulator
- Outputs N 48-bit biased results

Scaling Math:

```
Accumulator: Q14.33 format (products summed)
Bias: Q4.11 format (16 bits)
Scaled bias = bias << 22 bits (to align decimal points)

Actually, product is Q8.22, so:
Scaled bias = bias << 11 bits (22 - 11)
```

Key Behaviors:

- When enabled: output[i] = input[i] + (bias[i] << 11)
- When disabled: output[i] = input[i] (pass-through)
- Only first m_dim elements are processed

Timing: 1 cycle (registered output)

Interfaces:

- Inputs: clk, rst, enable, data_in[N×48-1:0], bias_in[N×16-1:0], valid_in, m_dim[5:0]
- Outputs: data_out[N×48-1:0], valid_out

8.3.2 activation_unit

Purpose: Applies activation function to all elements in parallel.

Supported Activations:

Code	Name	Formula	Use Case
00	LINEAR	y = x	Output layer
01	RELU	y = max(0, x)	Hidden layers
10	LEAKY_RELU	y = x>0 ? x : x/8	Prevent dying neurons
11	RESERVED	y = x	Future use

Function:

- Receives N 48-bit values
- Applies selected activation to each
- Outputs N 48-bit activated values
- Only processes first m_dim elements

Key Behaviors:

- LINEAR: Pass through unchanged
- RELU: Zero if negative (check sign bit)
- LEAKY_RELU: Arithmetic right shift by 3 if negative (divide by 8)

Timing: 1 cycle (registered output)

Interfaces:

- Inputs: clk, rst, data_in[N×48-1:0], valid_in, activation_sel[1:0], m_dim[5:0]
- Outputs: data_out[N×48-1:0], valid_out

8.3.3 output_saturator

Purpose: Converts 48-bit accumulated values to 16-bit Q4.11 format with rounding and saturation.

Function:

- Receives N 48-bit values
- Adds rounding constant (1024 = 2^10 = 0.5 LSB)
- Arithmetic shift right by 11 bits
- Saturates to [-32768, +32767] range
- Sets overflow flags for saturated values

Algorithm:

```
For each element i:
  1. rounded = data_in[i] + 1024
  2. shifted = rounded >>> 11 (arithmetic shift)
  3. if shifted > 32767: output = 32767, overflow = 1
     elif shifted < -32768: output = -32768, overflow = 1
     else: output = shifted[15:0], overflow = 0
```

Key Behaviors:

- Rounding prevents systematic bias from truncation
- Saturation prevents wrap-around on overflow
- Per-element overflow flags for diagnostics

Timing: 1 cycle (registered output)

Interfaces:

- Inputs: clk, rst, data_in[N×48-1:0], valid_in, m_dim[5:0]
- Outputs: data_out[N×16-1:0], valid_out, overflow[N-1:0]

8.3.4 postprocess_subsystem (Wrapper)

Purpose: Chains bias_adder → activation_unit → output_saturator with registered pipeline.

Pipeline:

```
Cycle 0: data_in arrives
        ↓
Cycle 1: bias_adder output
        ↓
Cycle 2: activation_unit output
        ↓
Cycle 3: output_saturator output (data_out)

Total latency: 3 cycles
```

Function:

- Instantiates and chains all three processing stages
- Manages valid signal propagation through pipeline
- Combines overflow flags from saturator

Interfaces:

- Inputs: clk, rst, data_in[N×48-1:0], bias_in[N×16-1:0], valid_in, bias_enable, activation_sel[1:0], m_dim[5:0]
- Outputs: data_out[N×16-1:0], valid_out, overflow[N-1:0]

---

9. Subsystem 4: Control Subsystem

9.1 Purpose

The control subsystem manages **configuration, address generation, and operation sequencing**. It orchestrates all other subsystems through a master state machine.

9.2 Hierarchy

```
control_subsystem
│
├── configuration_registers (1 instance)
│   └── Store and validate config
│
├── address_generator (1 instance)
│   └── Generate memory addresses
│
└── main_fsm (1 instance)
    └── Master state machine

Total: 3 module instances
```

9.3 Module Descriptions

9.3.1 configuration_registers

Purpose: Stores and validates runtime configuration parameters.

Stored Configuration:

Parameter	Width	Valid Range	Description
m_dim	6 bits	1 to MAX_DIM	Output dimension (rows)
k_dim	6 bits	1 to MAX_DIM	Input dimension (cols)
op_type	2 bits	0-2	Operation type
activation	2 bits	0-2	Activation function
bias_en	1 bit	0-1	Enable bias addition

Operation Types:

Code	Name	Description
00	OP_MATVEC	Y = W × X + B (forward pass)
01	OP_MATVEC_T	Y = W^T × X (backward pass)
10	OP_OUTER	Y = X × Z^T (weight update)
11	OP_NOOP	No operation

Function:

- Validates inputs on write (dimensions in range)
- Latches configuration on write_en
- Provides stable outputs for other subsystems
- Flags invalid configurations

Key Behaviors:

- Valid config: All parameters latched, config_valid = 1
- Invalid config: Parameters not latched, config_error = 1

Timing: 1 cycle to latch configuration

Interfaces:

- Inputs: clk, rst, write_en, m_dim_in[5:0], k_dim_in[5:0], op_type_in[1:0], activation_in[1:0], bias_en_in
- Outputs: m_dim[5:0], k_dim[5:0], op_type[1:0], activation[1:0], bias_en, config_valid, config_error

---

9.3.2 address_generator

Purpose: Generates sequential or transposed addresses for weight memory access.

Function:

- Generates row and column addresses for weight access
- Supports normal mode (row-major) and transpose mode (column-major)
- Tracks completion of rows and entire matrix
- Used primarily during weight loading

Address Patterns:

Mode	Sequence	Use Case
Normal	(0,0)→(0,K-1)→(1,0)→...→(M-1,K-1)	Forward pass
Transpose	(0,0)→(M-1,0)→(0,1)→...→(M-1,K-1)	Backward pass

Key Behaviors:

- On start: Initialize counter to 0
- Each cycle: Output address, advance counter
- row_done: Signal end of current row
- all_done: Signal end of all addresses

Timing: 1 address per cycle

Interfaces:

- Inputs: clk, rst, start, m_dim[5:0], k_dim[5:0], transpose
- Outputs: row_addr[5:0], col_addr[5:0], addr_valid, row_done, all_done, busy

---

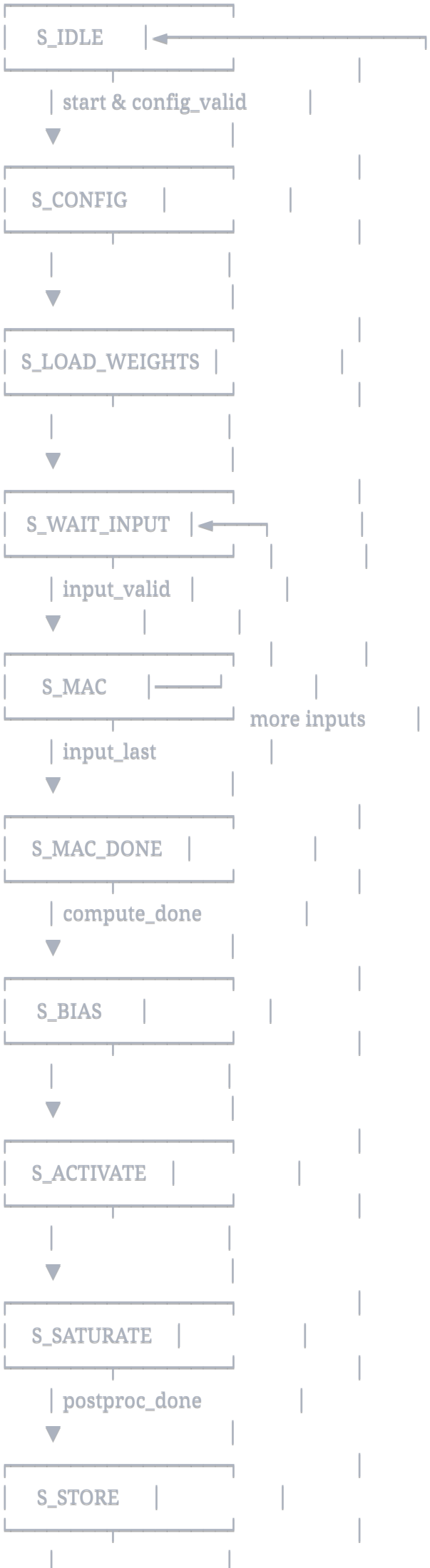
9.3.3 main_fsm

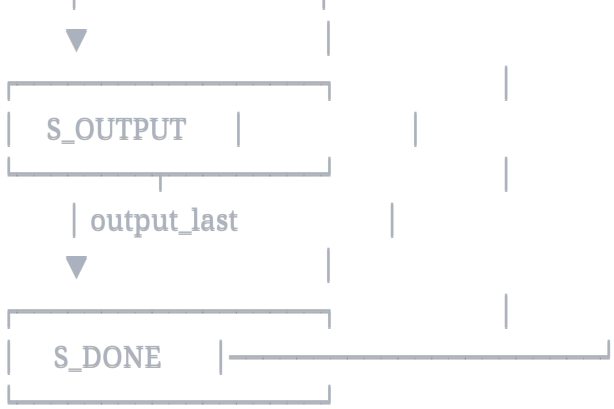
Purpose: Master state machine controlling entire matrix operation.

States:

State	Code	Description
S_IDLE	0000	Waiting for start command
S_CONFIG	0001	Load and validate configuration
S_LOAD_WEIGHTS	0010	Read weight row from memory
S_WAIT_INPUT	0011	Wait for input element
S_MAC	0100	MAC accumulating (K cycles)
S_MAC_DONE	0101	Wait for adder tree completion
S_BIAS	0110	Add bias to results
S_ACTIVATE	0111	Apply activation function
S_SATURATE	1000	Saturate outputs
S_STORE	1001	Store to output buffer
S_OUTPUT	1010	Stream outputs externally
S_DONE	1011	Operation complete

State Transition Diagram:


S_IDLE
start & config_valid
S_CONFIG
S_LOAD_WEIGHTS
S_WAIT_INPUT
input_valid
S_MAC
more inputs
input_last
S_MAC_DONE
compute_done
S_BIAS
S_ACTIVATE
S_SATURATE
postproc_done
S_STORE



Control Signal Generation:

State	Key Control Signals
S_IDLE	All signals low
S_CONFIG	config_load = 1, mac_clear = 1, input_clear = 1
S_LOAD_WEIGHTS	weight_read_en = 1, weight_transpose = (op=MATVEC_T)
S_WAIT_INPUT	input_ready = 1
S_MAC	mac_enable = 1, input_ready = 1
S_MAC_DONE	(wait for compute_done)
S_BIAS	bias_enable = (op=MATVEC), postproc_enable = 1
S_ACTIVATE	postproc_enable = 1
S_SATURATE	postproc_enable = 1
S_STORE	output_load = 1
S_OUTPUT	output_read_en = output_ready
S_DONE	done = 1

Timing: Variable based on operation dimensions

Interfaces:

- Inputs: clk, rst, start, config_valid, input_valid, input_last, compute_done, postproc_done, output_ready, output_last, m_dim[5:0], k_dim[5:0], op_type[1:0]
- Outputs: state_out[3:0], config_load, weight_read_en, weight_transpose, input_ready, input_clear, mac_clear, mac_enable, bias_enable, postproc_enable, output_load, output_read_en, busy, done

---

9.3.4 control_subsystem (Wrapper)

Purpose: Wraps all control modules with unified interface.

Function:

- Instantiates config_regs, addr_gen, main_fsm
- Routes configuration to all subsystems
- Generates all control signals
- Aggregates status signals

Interfaces: Combined interfaces of all three sub-modules plus inter-subsystem signals

---

10. Top Level Integration

10.1 matrix_engine_top

Purpose: Top-level module that instantiates and connects all four subsystems.

Parameters:

Generic	Type	Default	Description
MAX_DIM	integer	8	Maximum matrix dimension (N)
DATA_WIDTH	integer	16	Data width (Q4.11)
ACCUM_WIDTH	integer	48	Accumulator width
FRAC_BITS	integer	11	Fractional bits

10.2 External Interface

Configuration Ports

Direction	Signal	Width	Description
in	clk	1	System clock
in	rst	1	Synchronous reset
in	cfg_start	1	Start operation
in	cfg_m_dim	6	Output dimension [1..N]
in	cfg_k_dim	6	Input dimension [1..N]
in	cfg_op_type	2	Operation (00=MatVec, 01=MatVec_T, 10=Outer)
in	cfg_activation	2	Activation (00=Lin, 01=ReLU, 10=LeakyReLU)
in	cfg_bias_en	1	Enable bias addition

Weight Loading Ports

Direction	Signal	Width	Description
in	wgt_write_en	1	Write enable
in	wgt_write_addr	8	Weight address
in	wgt_write_data	16	Weight value
in	wgt_write_bias	1	Writing to bias (vs weight)

Input Streaming Ports

Direction	Signal	Width	Description
in	inp_data	16	Input element (Q4.11)
in	inp_index	6	Element index [0..K-1]
in	inp_valid	1	Input is valid
in	inp_last	1	Last input element
out	inp_ready	1	Engine ready for input

Output Streaming Ports

Direction	Signal	Width	Description
out	out_data	16	Output element (Q4.11)
out	out_index	6	Element index [0..M-1]
out	out_valid	1	Output is valid
out	out_last	1	Last output element
in	out_ready	1	External ready to receive

Status Ports

Direction	Signal	Width	Description
out	busy	1	Operation in progress
out	done	1	Operation complete
out	overflow	1	Any overflow occurred
out	error	1	Configuration error

10.3 Internal Interconnections

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                          matrix_engine_top                                   │
│                                                                              │
│  ┌──────────────────────────────────────────────────────────────────────────┐ │
│  │                        control_subsystem                                 │ │
│  │                                                                          │ │
│  │  Configuration, Address Generation, Master FSM                           │ │
│  │                                                                          │ │
│  └──────────────────────────────────┬───────────────────────────────────────┘ │
│                                     │                                        │
│      ┌──────────────────────────────┼──────────────────────────────┐         │
│      │                              │                              │         │
│      ▼                              ▼                              ▼         │
│  ┌──────────────────┐      ┌──────────────────┐      ┌──────────────────┐   │
│  │   memory_        │      │  compute_        │      │ postprocess_     │   │
│  │  subsystem       │      │  subsystem       │      │  subsystem       │   │
│  │                  │      │                  │      │                  │   │
│  │ weight_memory    ├──────┤                  │      │                  │   │
│  │                  │ weights │ mac_array      │      │ bias_adder       │   │
│  │                  │      │                  │      │                  │   │
│  │ input_buffer     ├──────┤ mac_row[N]       ├──────┤ activation       │   │
│  │                  │ inputs │ accum           │      │                  │   │
│  │                  │      │ mac_unit         │      │ saturator        │   │
│  │ output_buffer    ◄──────┤ adder_tree       ◄──────┤                  │   │
│  │                  │ results │                │      │                  │   │
│  └──────────────────┘      └──────────────────┘      └──────────────────┘   │
│                                                                              │
└──────────────────────────────────────────────────────────────────────────────┘
```

Data Flow:

1. Weights loaded to weight_memory via write interface
2. Inputs stream into input_buffer via input interface
3. weight_memory provides row to mac_array
4. input_buffer provides vector to mac_array
5. mac_array produces accumulated results
6. postprocess_subsystem adds bias, applies activation, saturates
7. Results loaded to output_buffer
8. output_buffer streams to external interface

Control Flow:

1. External start triggers main_fsm
2. FSM sequences through states
3. Control signals enable each subsystem at appropriate time
4. Done signal indicates completion

---

11. Operations and Data Flow

11.1 Operation: MatVec (Forward Pass)

Purpose: Compute Y = W × X + B

Configuration:

- op_type = 00 (OP_MATVEC)
- m_dim = number of outputs (neurons in layer)
- k_dim = number of inputs (features or previous layer outputs)
- activation = layer's activation function
- bias_en = 1

Data Flow:

```
1. Load weights W[M×K] to weight_memory
2. Load bias B[M] to weight_memory
3. Stream input X[K] to input_buffer
4. mac_array computes: sum[m] = Σ(W[m][k] × X[k]) for k=0..K-1
5. bias_adder computes: Y'[m] = sum[m] + B[m]
6. activation_unit computes: Y[m] = f(Y'[m])
7. output_saturator produces: Y_out[m] in Q4.11
8. Stream Y_out[M] from output_buffer
```

Example: Layer 1 (4→8)

- m_dim = 8 (8 neurons)
- k_dim = 4 (4 inputs)
- Result: 8 activations

11.2 Operation: MatVec_T (Backward Pass)

Purpose: Compute δ_prev = W^T × δ_curr (error propagation)

Configuration:

- op_type = 01 (OP_MATVEC_T)
- m_dim = number of previous layer neurons (outputs)
- k_dim = number of current layer neurons (inputs)
- activation = 00 (LINEAR - no activation during backprop)
- bias_en = 0 (no bias in error propagation)

Data Flow:

```
1. W already loaded (use transpose from weight_memory)
2. Stream delta δ_curr[K] to input_buffer
3. mac_array uses W^T: sum[m] = Σ(W^T[m][k] × δ_curr[k])
4. No bias addition (disabled)
5. Linear activation (pass through)
6. output_saturator produces: δ_prev[m] in Q4.11
7. Stream δ_prev[M] from output_buffer
```

Example: Error from L2 (4 neurons) to L1 (8 neurons)

- m_dim = 8 (L1 neurons)
- k_dim = 4 (L2 neurons)
- W^T is 8×4 (transpose of 4×8)

11.3 Operation: OuterProduct (Weight Update)

Purpose: Compute gradient = δ × a^T (weight gradient)

Configuration:

- op_type = 10 (OP_OUTER)
- m_dim = number of current layer neurons
- k_dim = number of previous layer activations
- activation = 00 (LINEAR)
- bias_en = 0

Data Flow:

```
1. Stream delta δ[M] as "input vector"
2. Stream previous activations a[K] as "weight column"
3. Outer product: grad[m][k] = δ[m] × a[k]
4. No bias, linear activation
5. output_saturator produces gradient matrix
```

Note: OuterProduct generates M×K values, which may require M passes of K outputs each, or special handling.

11.4 Timing Analysis

Single MatVec Operation (M=8, K=8)

Phase	Cycles	Description
Config	1	Latch configuration
Weight read	1	Read weight row (parallel)
Input stream	K=8	One input per cycle
Adder tree	2	Pipeline stages
Postprocess	3	Bias + activation + saturation
Output stream	M=8	One output per cycle
TOTAL	**23**	Cycles per MatVec

Complete Forward Pass (4-8-4-1 Network)

Layer	M	K	Cycles
L1: 4→8	8	4	1+1+4+2+3+8 = 19
L2: 8→4	4	8	1+1+8+2+3+4 = 19
L3: 4→1	1	4	1+1+4+2+3+1 = 12
TOTAL			**50 cycles**

Comparison to Sequential Design

Metric	Sequential	Matrix Engine	Improvement
Forward pass	110 cycles	50 cycles	**2.2×**
Full training	210 cycles	105 cycles	**2.0×**

---

12. Zero Padding Strategy

12.1 Concept

The matrix engine is designed for MAX_DIM × MAX_DIM operations. Smaller dimensions are handled by **zero-padding** the unused positions.

Key Insight: 0 × anything = 0, and adding 0 doesn't change the sum.

12.2 Example: 4×3 on 8×8 Engine

Problem: Compute 4×3 matrix-vector product using 8×8 hardware.

Weight Matrix Setup:

```
Actual 4×3 weights:        Padded 8×8 weights:

┌                 ┐          ┌                                 ┐
│ w00 w01 w02 │          │ w00 w01 w02  0   0   0   0   0 │
│ w10 w11 w12 │    →      │ w10 w11 w12  0   0   0   0   0 │
│ w20 w21 w22 │          │ w20 w21 w22  0   0   0   0   0 │
│ w30 w31 w32 │          │ w30 w31 w32  0   0   0   0   0 │
└                 ┘          │  0   0   0   0   0   0   0   0 │
                   │ 0   0   0   0   0   0   0   0 │
                   │ 0   0   0   0   0   0   0   0 │
                   │ 0   0   0   0   0   0   0   0 │
                   └                                 ┘
```

Input Vector Setup:

```
Actual 3-element input:     Padded 8-element input:

┌       ┐                 ┌                                 ┐
│ x0  │                 │ x0  x1  x2  0   0   0   0   0 │
│ x1  │          →       └                                 ┘
│ x2  │
└       ┘
```

Computation:

```
For row 0:
  sum = w00×x0 + w01×x1 + w02×x2 + 0×0 + 0×0 + 0×0 + 0×0 + 0×0
      = w00×x0 + w01×x1 + w02×x2  ✓ Correct!

For row 4 (padded):
  sum = 0×x0 + 0×x1 + 0×x2 + 0×0 + ... = 0  (ignored)
```

12.3 Control Logic for Zero-Padding

Input Buffer

```
Function: Provide zero-padded parallel output

for i in 0 to MAX_DIM-1:
  if i < k_dim:
    data_out[i] = buffer[i]
  else:
    data_out[i] = 0  // Zero pad
```

MAC Enable Masking

```
Function: Disable MACs beyond k_dim

for i in 0 to MAX_DIM-1:
  mac_enable[i] = global_enable AND (i < k_dim)
```

Row Active Masking

```
Function: Ignore results beyond m_dim

for i in 0 to MAX_DIM-1:
  row_valid[i] = (i < m_dim)
```

Output Buffer

```
Function: Only output m_dim elements

num_outputs = m_dim (not MAX_DIM)
last = (index == m_dim - 1)
```

12.4 Benefits of Zero-Padding

1. **Single Hardware Configuration**: One engine handles all layer sizes
2. **No Reconfiguration Overhead**: Just change m_dim, k_dim registers
3. **Mathematically Correct**: Zero contributions don't affect results
4. **Simple Control Logic**: Just compare indices to dimensions
5. **Efficient for Common Sizes**: Most neural networks have layers ≤ 32 neurons

12.5 Efficiency Analysis

Layer Size	Active MACs	Total MACs (8×8)	Utilization
8×8	64	64	100%
4×8	32	64	50%
8×4	32	64	50%
4×4	16	64	25%
1×4	4	64	6.25%

Observation: Smaller layers have lower utilization, but overall speedup is still significant due to parallel processing.

---

13. Development Plan

13.1 Phase Overview

Phase	Duration	Focus	Deliverables
Phase 1	Week 1	Compute Primitives	mac_unit, adder_tree tested
Phase 2	Week 2	Compute Array	mac_row, mac_array working
Phase 3	Week 3	Memory System	All buffers and weight memory
Phase 4	Week 4	Postprocessing	Bias, activation, saturation
Phase 5	Week 5	Control Logic	FSM and configuration
Phase 6	Week 6	Integration	Complete engine tested

13.2 Phase 1: Compute Primitives (Week 1)

Goals

- Implement and verify fundamental computation units
- Establish fixed-point arithmetic correctness

Tasks

Day	Task	Deliverable
1	Design mac_unit	VHDL entity + architecture
2	Create mac_unit testbench	Comprehensive test vectors
3	Verify mac_unit	Simulation passing
4	Design adder_tree	VHDL entity + architecture
5	Create adder_tree testbench	Test various input patterns
6	Verify adder_tree	Simulation passing
7	Review and document	Phase 1 report

Verification Focus

- Fixed-point multiplication correctness
- Accumulator overflow behavior
- Adder tree reduction accuracy
- Timing and latency

Exit Criteria

- mac_unit: 100% test pass rate
- adder_tree: All input patterns produce correct sum
- No timing violations at 100 MHz

13.3 Phase 2: Compute Array (Week 2)

Goals

- Build up from primitives to full MAC array
- Verify row-level and array-level operation

Tasks

Day	Task	Deliverable
1	Design mac_row	VHDL entity + architecture
2	Create mac_row testbench	Dot product verification
3	Verify mac_row	Known vector tests
4	Design mac_array	VHDL entity + architecture
5	Create mac_array testbench	Matrix-vector tests
6	Verify mac_array	Python golden model comparison
7	Create compute_subsystem wrapper	Integration verification

Verification Focus

- Single row dot product correctness
- All rows computing in parallel
- Zero-padding (k_dim masking)
- Row masking (m_dim)

Exit Criteria

- mac_row: Correct dot products for various k_dim
- mac_array: Matches Python NumPy results
- compute_subsystem: Clean interface working

13.4 Phase 3: Memory System (Week 3)

Goals

- Implement all memory modules
- Verify serial-parallel conversions

Tasks

Day	Task	Deliverable
1	Design weight_memory	VHDL + addressing
2	Test weight_memory	Write/read verification
3	Verify transpose access	W^T read correctness
4	Design input_buffer	Serial-in, parallel-out
5	Design output_buffer	Parallel-in, serial-out
6	Test all buffers	Buffer testbench
7	Create memory_subsystem wrapper	Integration test

Verification Focus

- Weight memory address mapping
- Automatic transpose generation
- Buffer serial-parallel conversion
- Zero-padding in input buffer

Exit Criteria

- Weight memory: All addresses accessible
- Transpose: W^T[i][j] == W[j][i]
- Buffers: Correct serial↔parallel conversion

13.5 Phase 4: Postprocessing (Week 4)

Goals

- Implement all postprocessing stages
- Verify activation functions and saturation

Tasks

Day	Task	Deliverable
1	Design bias_adder	VHDL + scaling
2	Test bias_adder	Various bias values
3	Design activation_unit	Linear, ReLU, Leaky
4	Test activation_unit	All activation modes
5	Design output_saturator	Rounding + saturation
6	Test saturation	Overflow cases
7	Create postprocess_subsystem	Pipeline integration

Verification Focus

- Bias scaling correctness (bit alignment)
- ReLU: max(0, x)
- Leaky ReLU: x>0 ? x : x/8
- Saturation to Q4.11 range
- Pipeline timing (3 cycle latency)

Exit Criteria

- Bias adds correctly to accumulator
- Activations match expected behavior
- No incorrect saturation/overflow
- Pipeline latency = 3 cycles

13.6 Phase 5: Control Logic (Week 5)

Goals

- Implement FSM and configuration
- Verify complete operation sequencing

Tasks

Day	Task	Deliverable
1	Design configuration_registers	VHDL + validation
2	Design address_generator	Normal + transpose
3	Design main_fsm	All 12 states
4	Test FSM state transitions	State coverage
5	Test control signal generation	Signal timing
6	Create control_subsystem wrapper	Integration
7	Full control verification	End-to-end sequencing

Verification Focus

- Configuration validation (m_dim, k_dim in range)
- FSM transitions (all paths covered)
- Control signal timing (correct cycles)
- Handshaking (ready/valid protocols)

Exit Criteria

- All FSM states reachable
- Correct control signals per state
- Proper handshaking with data paths

13.7 Phase 6: Integration (Week 6)

Goals

- Integrate all subsystems
- Verify complete neural network operations

Tasks

Day	Task	Deliverable
1	Create matrix_engine_top	All subsystems connected
2	Basic integration test	Simple MatVec
3	Test all operations	MatVec, MatVec_T, Outer
4	Test all activations	Linear, ReLU, Leaky
5	Test edge cases	m=1, k=1, overflow
6	Neural network test	AND/XOR/Iris
7	Performance measurement	Cycle counts, throughput

Verification Focus

- All subsystems communicate correctly
- All operations produce correct results
- Edge cases handled properly
- Neural network training converges

Exit Criteria

- All operations working
- Iris classification >80% accuracy
- Performance meets targets
- No synthesis errors

---

14. Testing Strategy

14.1 Test Hierarchy

```
Level 4: System Tests
  └── Neural network training (Iris, AND, XOR)

Level 3: Integration Tests
  └── matrix_engine_top complete operations

Level 2: Subsystem Tests
  ├── compute_subsystem matrix-vector products
  ├── memory_subsystem load/store operations
  ├── postprocess_subsystem pipeline
  └── control_subsystem FSM sequences

Level 1: Module Tests
  ├── mac_unit multiply-accumulate
  ├── adder_tree reduction
  ├── mac_row dot product
  ├── weight_memory addressing
  ├── input_buffer serial→parallel
  ├── output_buffer parallel→serial
  ├── bias_adder scaling
  ├── activation_unit functions
  ├── output_saturator rounding
  └── main_fsm state transitions
```

14.2 Test Types

Unit Tests (Per Module)

Test Type	Description
Functional	Correct output for valid inputs
Boundary	Edge cases (min, max values)
Reset	Proper initialization after reset
Enable	Correct enable/disable behavior
Overflow	Saturation on extreme values

Integration Tests (Per Subsystem)

Test Type	Description
Interface	Correct signals between modules
Timing	Proper latency and throughput
Pipeline	Data flows through all stages
Control	Handshaking works correctly

System Tests (Complete Engine)

Test Type	Description
MatVec	Y = W×X+B correctness
MatVec_T	Y = W^T×X correctness
OuterProduct	Y = X×Z^T correctness
Dimensions	Various m_dim, k_dim combinations
Activations	All activation functions
Neural Network	Full training convergence

14.3 Golden Model

A **Python golden model** using NumPy provides reference outputs:

```python
class MatrixEngineGolden:
    def __init__(self, max_dim=8, frac_bits=11):
        self.max_dim = max_dim
        self.frac_bits = frac_bits
        self.scale = 2 ** frac_bits

    def to_fixed(self, val):
        """Convert float to Q4.11 fixed-point"""
        return int(round(val * self.scale))

    def from_fixed(self, val):
        """Convert Q4.11 fixed-point to float"""
        return val / self.scale

    def saturate(self, val):
        """Saturate to 16-bit signed range"""
        if val > 32767:
            return 32767
        elif val < -32768:
            return -32768
        return val

    def matvec(self, W, X, B, activation='linear'):
        """Compute Y = activation(W @ X + B)"""
        # Fixed-point computation...
        pass
```

14.4 Test Vector Generation

Random Test Vectors

- Random weights in [-2.0, +2.0]
- Random inputs in [-1.0, +1.0]
- All dimension combinations

Specific Test Vectors

- Identity matrix (output = input)
- All zeros (output = bias only)
- All ones (test accumulation)
- Maximum values (test saturation)
- Minimum values (test negative handling)

14.5 Coverage Goals

Coverage Type	Target
Statement	100%
Branch	100%
FSM State	All states visited
FSM Transition	All transitions exercised
Boundary	All limits tested

---

15. Resource Estimates

15.1 Per-Module Estimates (8×8 Engine)

Module	DSPs	LUTs	FFs
mac_unit (×64)	64	0	768
adder_tree (×8)	0	672	288
mac_row (×8)	(included above)	64	64
mac_array	0	64	32
weight_memory	0	136	2176
input_buffer	0	64	144
output_buffer	0	64	160
bias_adder	0	128	96
activation_unit	0	64	48
output_saturator	0	256	144
configuration_registers	0	32	32
address_generator	0	48	32
main_fsm	0	128	64
TOTAL	**64**	**~1720**	**~4048**

15.2 Subsystem Totals

Subsystem	DSPs	LUTs	FFs	BRAM
compute_subsystem	64	~800	~1152	0
memory_subsystem	0	~264	~2480	0*
postprocess_subsystem	0	~448	~288	0
control_subsystem	0	~208	~128	0
TOTAL	**64**	**~1720**	**~4048**	**0**

*Weight memory can use distributed RAM or BRAM depending on implementation choice.

15.3 Target FPGA Utilization

Artix-7 35T (xc7a35t)

Resource	Available	Used (8×8)	Utilization
DSP48E1	90	64	71%
LUTs	20,800	~1,720	8%
FFs	41,600	~4,048	10%
BRAM	50 (18Kb)	1	2%

Conclusion: 8×8 engine fits comfortably on entry-level FPGA.

15.4 Scaling to 32×32

Resource	8×8 Engine	32×32 Engine	Increase
DSPs	64	1024	16×
LUTs	~1,720	~19,000	11×
FFs	~4,048	~50,000	12×
BRAM	1	8	8×

32×32 Requirements: Requires larger FPGA (e.g., Artix-7 100T or Kintex-7)

---

16. Performance Analysis

16.1 Latency Breakdown

Single MatVec (M=8, K=8)

Phase	Cycles	Cumulative
Start/Config	1	1
Weight read	1	2
Input streaming	8	10
Adder tree	2	12
Bias addition	1	13
Activation	1	14
Saturation	1	15
Output streaming	8	23
TOTAL	**23**	

Formula

```
Latency = 1 + 1 + K + 2 + 3 + M
        = K + M + 7 cycles
```

16.2 Throughput Analysis

Metric	Value
Clock frequency	100 MHz
Cycle time	10 ns
MatVec latency (8×8)	23 cycles = 230 ns
MatVec throughput	4.35 M ops/sec
MAC operations per MatVec	64
Peak MACs/sec	278 GMACs/sec (theoretical)
Effective MACs/sec	64/23 × 100M = 278 MMACs/sec

16.3 Comparison with Sequential Design

Current Sequential Design

Layer	Neurons	Cycles
L1 (4→8)	8	~50
L2 (8→4)	4	~40
L3 (4→1)	1	~20
Total Forward		**~110**
Backward + Update		~100
Total Training		**~210**

Matrix Engine Design

Layer	M × K	Cycles
L1 (4→8)	8×4	4+8+7 = 19
L2 (8→4)	4×8	8+4+7 = 19
L3 (4→1)	1×4	4+1+7 = 12
Total Forward		**~50**
Backward + Update		~55
Total Training		**~105**

Speedup

Metric	Sequential	Matrix Engine	Speedup
Forward pass	110 cycles	50 cycles	**2.2×**
Full training	210 cycles	105 cycles	**2.0×**
Samples/sec @ 100MHz	476K	952K	**2.0×**

16.4 Efficiency Analysis

MAC Utilization

Layer	Active MACs	Total (64)	Utilization
L1 (8×4)	32	64	50%
L2 (4×8)	32	64	50%
L3 (1×4)	4	64	6.25%
Average	~23	64	**~36%**

Energy Efficiency

Parallel processing completes faster, allowing:

- Earlier clock gating
- Lower average power
- Better performance/watt

---

17. Future Enhancements

17.1 Near-Term Improvements

17.1.1 Double Buffering

- Add second input/output buffer
- Overlap data transfer with computation
- Potential 20-30% throughput increase

17.1.2 Weight Caching

- Cache weights for frequently-used layers
- Reduce weight loading overhead
- Useful for inference (fixed weights)

17.1.3 Batch Processing

- Process multiple samples simultaneously
- Better utilization of padded dimensions
- Improved training throughput

17.2 Medium-Term Enhancements

17.2.1 Tiling for Large Matrices

- Handle matrices larger than MAX_DIM
- Tile computation into MAX_DIM × MAX_DIM blocks
- Accumulate partial results

17.2.2 Additional Activation Functions

- Sigmoid (lookup table or CORDIC)
- Tanh (lookup table or CORDIC)
- Softmax (for multi-class classification)

17.2.3 Batch Normalization

- Add batch norm layer support
- Running mean/variance tracking
- Inference and training modes

17.3 Long-Term Roadmap

17.3.1 Convolution Support

- Extend to 2D convolutions
- Im2col transformation
- Kernel tiling

17.3.2 Quantization Support

- INT8 inference mode
- Mixed-precision training
- Dynamic quantization

17.3.3 Multi-Engine Parallelism

- Multiple matrix engines
- Layer-parallel execution
- Pipeline across layers

---

18. Appendices

Appendix A: Signal Naming Conventions

Prefix/Suffix	Meaning
`_in`	Input port
`_out`	Output port
`_valid`	Data is valid
`_ready`	Ready to receive
`_last`	Last item in sequence
`_en`	Enable signal
`_clr`	Clear/reset signal
`l1_`, `l2_`, `l3_`	Layer 1, 2, 3
`wgt_`	Weight-related
`inp_`	Input-related
`out_`	Output-related
`cfg_`	Configuration

Appendix B: Fixed-Point Conversion Reference

Q4.11 Format

Decimal	Q4.11 Binary	Q4.11 Integer
1.0	0000.10000000000	2048
0.5	0000.01000000000	1024
0.25	0000.00100000000	512
-1.0	1111.10000000000	-2048
15.999	0111.11111111111	32767
-16.0	1000.00000000000	-32768

Conversion Formulas

```
Float to Q4.11: fixed = round(float × 2048)
Q4.11 to Float: float = fixed / 2048
```

Appendix C: FSM State Encoding

State	Binary	Hex
S_IDLE	0000	0x0
S_CONFIG	0001	0x1
S_LOAD_WEIGHTS	0010	0x2
S_WAIT_INPUT	0011	0x3
S_MAC	0100	0x4
S_MAC_DONE	0101	0x5
S_BIAS	0110	0x6
S_ACTIVATE	0111	0x7
S_SATURATE	1000	0x8
S_STORE	1001	0x9
S_OUTPUT	1010	0xA
S_DONE	1011	0xB

Appendix D: Error Codes

Code	Meaning
0x00	No error
0x01	Invalid m_dim (0 or > MAX_DIM)
0x02	Invalid k_dim (0 or > MAX_DIM)
0x03	Invalid operation type
0x04	Overflow during computation
0x05	Timeout waiting for input
0x06	Timeout waiting for output ready

Appendix E: Memory Map Summary

Weight Memory (8×8 Engine)

Address	Content
0x00-0x07	W[0][0..7]
0x08-0x0F	W[1][0..7]
0x10-0x17	W[2][0..7]
0x18-0x1F	W[3][0..7]
0x20-0x27	W[4][0..7]
0x28-0x2F	W[5][0..7]
0x30-0x37	W[6][0..7]
0x38-0x3F	W[7][0..7]
0x40-0x47	W^T[0][0..7]
0x48-0x4F	W^T[1][0..7]
0x50-0x57	W^T[2][0..7]
0x58-0x5F	W^T[3][0..7]
0x60-0x67	W^T[4][0..7]
0x68-0x6F	W^T[5][0..7]
0x70-0x77	W^T[6][0..7]
0x78-0x7F	W^T[7][0..7]
0x80-0x87	Bias[0..7]

Appendix F: File Structure

```
matrix_engine/
├── doc/
│   └── MATRIX_ENGINE_COMPLETE_SPECIFICATION.md (this file)
│
├── rtl/
│   ├── compute_subsystem/
│   │   ├── mac_unit.vhd
│   │   ├── adder_tree.vhd
│   │   ├── mac_row.vhd
│   │   ├── mac_array.vhd
│   │   └── compute_subsystem.vhd
│   │
│   ├── memory_subsystem/
│   │   ├── weight_memory.vhd
│   │   ├── input_buffer.vhd
│   │   ├── output_buffer.vhd
│   │   └── memory_subsystem.vhd
│   │
│   ├── postprocess_subsystem/
│   │   ├── bias_adder.vhd
│   │   ├── activation_unit.vhd
│   │   ├── output_saturator.vhd
│   │   └── postprocess_subsystem.vhd
│   │
│   ├── control_subsystem/
│   │   ├── configuration_registers.vhd
│   │   ├── address_generator.vhd
│   │   ├── main_fsm.vhd
│   │   └── control_subsystem.vhd
│   │
│   └── matrix_engine_top.vhd
│
├── sim/
│   ├── tb_mac_unit.vhd
│   ├── tb_adder_tree.vhd
│   ├── tb_mac_row.vhd
│   ├── tb_mac_array.vhd
│   ├── tb_weight_memory.vhd
│   ├── tb_input_buffer.vhd
│   ├── tb_output_buffer.vhd
│   ├── tb_bias_adder.vhd
│   ├── tb_activation_unit.vhd
│   ├── tb_output_saturator.vhd
│   ├── tb_main_fsm.vhd
│   ├── tb_compute_subsystem.vhd
│   ├── tb_memory_subsystem.vhd
│   ├── tb_postprocess_subsystem.vhd
│   ├── tb_control_subsystem.vhd
│   └── tb_matrix_engine_top.vhd
```

```
│
├── python/
│   ├── golden_model.py
│   ├── test_vector_gen.py
│   └── result_checker.py
│
└── constraints/
    └── timing.xdc
```

Document History

Version	Date	Author	Changes
1.0	Dec 2024	FPGA NN Project	Initial complete specification

End of Document